

Семинар по АКОС №7

Процессы

fork

```
#include <unistd.h>  
pid_t fork(void);
```

- Создаёт дочерний процесс - “копию” родительского (Copy-On-Write)
- В дочернем процессе возвращает 0, в родительском - pid ребёнка

Полезные команды

- `pstree -p` – дерево процессов
- `tree -d /proc` – тоже дерево процессов через /proc file system
- `strace -f ./a.out` – отслеживать дочерние процессы

Буферизация вывода

```
#include <stdio.h>
```

```
void setbuf(FILE *stream, char *buf);
```

```
void setbuffer(FILE *stream, char *buf, size_t size);
```

```
void setlinebuf(FILE *stream);
```

```
int setvbuf(FILE *stream, char *buf, int mode, size_t size);
```

Mode:

`_IONBF` unbuffered

`_IOLBF` line buffered

`_IOFBF` fully buffered

Изменяет режим буферизации

`fflush(stdout)` – зафлашить stdout (сбросить буффер в файл)

wait

```
#include <sys/types.h>  
#include <sys/wait.h>
```

```
pid_t wait(int *wstatus);
```

```
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

```
int waitid(idtype_t idtype, id_t id, siginfo_t *infor, int options);
```

Дождаться изменения статуса ребёнка.

Макросы для проверки статуса:

WIFEXITED(wstatus), WEXITSTATUS(wstatus), ...

EXEC

#include <unistd.h>

extern char **environ;

int execl(const char *pathname, const char *arg, ... /* (char*) NULL */);
int execlp(const char *file, const char *arg, ... /* (char *) NULL */);
int execl_e(const char *pathname, const char *arg, ... /*, (char *) NULL, char *const envp[] */);

int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);

l xor v = 1

l => переменное число аргументов

v => массив с аргументами

p or e

p => ищет пути как shell, в \$PATH

e => передать переменные окружения в формате VAR=VALUE

Подменяет текущий процесс

pthread

```
#include <pthread.h>
```

```
int pthread_create(pthread_t* thread, const pthread_attr_t *attr,  
void *(*start_routine) (void *), void *arg);
```

```
int pthread_join(pthread_t thread, void **retval);
```

Создать поток / дождаться завершения потока

readelf

- `readelf -h elf_file` - заголовки (headers) файла
- `readelf -S elf_file` - секции файла
- `readelf -s elf_file` - таблица символов
- `readelf -r elf_file` - таблицы релокаций
- `readelf -d elf_file` - динамические секции

objdump

- `objdump -M intel -d elf_file` - дизассемблирование кода в формате intel
- `objdump -x elf_file` - информация о всех секциях
- `objdump -s elf_file` - содержимое всех секций
- `objdump -t elf_file` - таблица символов
- `objdump -s -j.rodata elf_file` - посмотреть конкретную секцию